

Programming Paradigms

BSc Computer Science - Durham University

Matthew Emerson

October 2026

Contents

1	Module Introduction	1
1.1	Assessment	1
1.2	Timetable	1
2	Systems Programming	1
2.1	Version Control and UNIX	1
2.2	The Basics of C	1
2.3	Control Flow and Functions	3
2.4	Data Types and Structures	3
2.5	Advanced Bash Scripting	3
2.6	Pointers	3
2.7	Memory Management	3
2.8	Debugging	3
2.9	Scope and Recursion	3
2.10	Libraries	3
2.11	stdlib	3
2.12	C++	3
3	Functional Programming	3
4	Object Oriented Programming	3
4.1	Java Fundamentals	3
4.2	OOP with Classes and Objects	3
4.3	Encapsulation	3
4.4	Inheritance	3
4.5	Abstraction with Abstract Classes	3
4.6	Abstraction with Interfaces	3
4.7	Polymorphism	3
4.8	Java Collections	3
4.9	Best Practises	3
I	Appendix	4
5	Bibliography	4
5.1	Module Professors	4
5.2	References	4
5.3	List of Figures	4

1 Module Introduction

Programming Paradigms has the module code COMP2221 and is led by Dr Stuart James.

1.1 Assessment

This module is assessed through one piece of summative coursework and two final exams, these can be seen in Table 1. Additional formatives are also shown in italics.

Title	Opening	Due	Feedback	Professor	Weight
<i>Systems Programming</i>	<i>12th Oct</i>	<i>12th Nov</i>	<i>10th Dec</i>	<i>Dr Stuart James</i>	-
Systems Programming	23rd Oct	10th Dec	21st Jan	Dr Stuart James	50%
Object Oriented & Functional	May Exam	-	-	Ivkovic & Moses	50%

Table 1: Assessments for Programming Paradigms.

1.2 Timetable

This module is taught through two one-hour lectures and one two-hour computer class each week as seen below in Table 2.

Day	Time	Class Type	Location	Lecturer
Monday	11:00 - 12:00	Lecture	CLC202	Dr Stuart James
Monday	16:00 - 18:00	Computer Class	CB-LG001	n/a
Friday	14:00 - 15:00	Lecture	CLC202	Dr Stuart James

Table 2: Michaelmas Term Timetable for Programming Paradigms.

2 Systems Programming

This section is taught by Dr Stuart James. There is no official reading list for this module.

Definition 2.0.0.1: Systems programming involves the development of individual pieces of software that allow the entire system to function as a single unit.

2.1 Version Control and UNIX

2.1.1 Introduction to UNIX

2.2 The Basics of C

2.2.1 Basic File Setup

Definition 2.2.1.1: Pre-processor directives are lines which start with a `#` and are commands to the C pre-processor.

- If the directive is `#include`, and `<>` are used, then `/usr/include` is prioritised in UNIX by convention, whereas if `"` are used, then the current directory is prioritised.
- **Definition 2.2.1.1:** If the directive is `#define`, then this is a definition macro. These are used to provide definitions in code.
- **Definition 2.2.1.2:** You can also use directives for debugging without generating overheads, these are called conditionals. They are as shown in Table 3.

Conditional	Function
<code>#ifdef VAR</code>	Tests if VAR is <code>#defined</code> .
<code>#ifndef VAR</code>	Tests if VAR is not <code>#defined</code> .
<code>#else</code>	Runs if the original condition evaluates as false.
<code>#endif</code>	Ends the if testing condition.

Table 3: Different types of conditionals.

You can also put debugging code in the preamble, as seen in Algorithm 1.

```
#define MY_DEBUG // Define an identifier.
```

```
#ifdef MY_DEBUG
    assert(i > 0);
    printf("i is %d \n", i);
#endif
```

Algorithm 1: Debugging Code Example.

Definition 2.2.1.4: You can also use `#define` to define parameterised macros. These act like functions, and an example can be seen in Algorithm 2.

```
#define ADD(a,b) a + b
```

Algorithm 2: Parameterised Macro Example.

Definition 2.2.1.5: The main() function is the entry function for C programs. You can only have one of these.

2.2.2 Printf Parameters

Value	Resulting Format
<code>%d</code>	Signed decimal (int).
<code>%u</code>	Unsigned decimal.
<code>%o, %x</code>	Octal and hexadecimal.
<code>%l</code>	Long.
<code>%f</code>	Floating point, so <code>%1.2f</code> will give 3.14.
<code>%e</code>	Floating point (exponent form).
<code>%c, %s</code>	Character and string.

Table 4: First parameter of printf.

2.3 Control Flow and Functions

2.4 Data Types and Structures

2.5 Advanced Bash Scripting

2.6 Pointers

2.7 Memory Management

2.8 Debugging

2.9 Scope and Recursion

2.10 Libraries

2.11 stdlib

2.12 C++

3 Functional Programming

This section is taught by Dr Billy Moses Jr.. [Check for recommended reading.](#)

4 Object Oriented Programming

This section is taught by Dr Mladen Ivkovic. [Check for recommended reading.](#)

4.1 Java Fundamentals

4.2 OOP with Classes and Objects

4.3 Encapsulation

4.4 Inheritance

4.5 Abstraction with Abstract Classes

4.6 Abstraction with Interfaces

4.7 Polymorphism

4.8 Java Collections

4.9 Best Practises

Part I

Appendix

5 Bibliography

5.1 Module Professors

Module Lead: Doctor Stuart James

- Focus: Artificial intelligence, computer vision, human computer interaction, and digital humanities.
- Email: stuart.a.james@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Billy Moses Jr.

- Focus: Algorithms and biologically inspired computing.
- Email: william.k.moses-jr@durham.ac.uk
- Office Hours: Email to make an appointment.

Lecturer: Doctor Mladen Ivkovic

- Focus: Performance prediction and optimisation.
- Email: mladen.ivkovic@durham.ac.uk
- Office Hours: Email to make an appointment.

5.2 References

5.3 List of Figures